Notes

Designing Large Scale AI Systems

a.y. 2025/2026

Davide Donà - 264467 Andrea Blushi - 264468

April 2026

Contents

1 Introduction **1**
1.1 Large Language Models (LLMs) 1
1.2 Software 3.0 1
1.3 Roles of AI 1
1.4 Challenges 2

2 Prompting **3**
2.1 Anatomy of a Prompt 3
2.2 API Parameters 3
2.3 Prompting Strategies 4
2.3.1 Prompt Structure 4
2.3.2 Few-shot prompting 5
2.3.3 Chain-of-thought prompting 5

3 Testing **7**
3.1 Single Shot Prompts 7
3.2 Self-Critique 7
3.3 Feedback Loops 8
3.3.1 Human in the loop 8
3.3.2 Automatic Feedback Loops 9

4 Stateful Agents **11**
4.1 Stateless 11
4.2 Default Stateful Agents 11
4.3 Full History 12
4.4 Sliding Window 12
4.5 Summarization 12
4.6 Long-Term Memory 13
4.7 When to use which pattern 13

5 Experiments **14**
5.1 Comparing Models 14
5.2 Test Sets 14
5.2.1 Overfitting in the test set 15
5.2.2 Validation Sets 15
5.2.3 Cross Validation 15
5.2.4 Temporal Data 16
5.3 Experiment Design 16
5.3.1 Hyperparameter Tuning 16

6 Statistical Testing **18**
6.1 True metrics vs Estimated metrics 18
6.1.1 Confidence Intervals 18
6.2 Randomness 19
6.2.1 Resampling Methods 19
6.2.2 Bootstraping Methods 19
6.3 Evaluating Regression 19
6.3.1 Metrics 20
6.3.2 Bias and Variance 20
6.4 Evaluating Classification 20
6.4.1 Class Imbalance 20
6.4.2 Cost Imbalance 21
6.4.3 Confusion Matrix 21
6.4.4 Precision and Recall 21
6.4.5 TPR and FPR - ROC space 22
6.4.6 Plotting thesholds 22
6.4.7 Coverage Matrix 22
6.4.8 AUC 22
6.4.9 Precision-Recall Curve 23
6.4.10 Use of Metrics 23
6.5 No Free Lunch Theorem 23

1 Introduction

1.1 Large Language Models (LLMs)

1.2 Software 3.0

The term "Software 3.0," introduced by Andrej Karpathy, describes how AI is reshaping the way we build software.

- **Software 1.0**: explicit rules written by programmers (if/else logic, loops).
- **Software 2.0**: behavior learned from data (neural networks trained on examples).
- **Software 3.0**: behavior specified in natural language (LLMs following prompts).

1.3 Roles of AI

Over the past decade, AI has moved from a niche technology to a core part of software design. In practice, AI systems tend to play three main roles:

- **AI as a Coding Agent**: AI systems can write, refactor, and review code, accelerating software development. Their output is largely deterministic because it is traditional code that can be inspected, tested, and debugged.
- **AI as a Component**: In this case, AI is embedded within a larger system, and the surrounding architecture is designed around its capabilities and limitations. This is not entirely new: many traditional machine learning models already fit this pattern.
- **AI as a User**: AI systems can also act as users of other systems, because they can consume inputs and generate outputs through your interfaces. Today, system design must account for AI clients that can call APIs, read documentation, and operate on application data. This is introducing a new question: Who are you building for? Human users, AI users, or both?

To design responsibly, we should distinguish when AI is unnecessary, when it is simply convenient, and when it is a true system component. Those choices are important since they respond to the question "What are you building for?" and this mainly depends on how reniewable and fixed we want our outputs.

- **No AI**: Use traditional software when rules are clear, logic is straightforward, and outputs are deterministic and easy to test.
- **AI as a Convenience**: The problem and rules are well understood, but prompting is faster or simpler than writing code. Outputs are still mostly predictable and testable, but reliability checks remain important.

- **AI as a System Component**: Use AI when behavior is too complex or context-dependent to encode with explicit rules. In these cases, conventional testing is harder because expected outputs are open-ended, and the value comes from handling ambiguity (for example, creative generation and open-ended Q&A).

This course focuses primarily on AI as a system component, while still covering the other two cases. In this context, we want to analyze how reviewable and fixed outputs are, and for which kind of users we are building for.

1.4 Challenges

Creating AI systems is more complex than traditional software because of the following challenges:

- **Getting a single call right**: LLMs are not deterministic, and their outputs can vary widely based on the prompt, which needs to be accurately designed to get the desired output.
- **Managing a conversation**: When building a system that interacts with an LLM, you need to manage the context across the conversation, which can make the costs grow rapidly.
- **Long-term memory**: LLMs have limited context windows, so you need to design strategies for storing and retrieving information (like preferences or conversation history) over time.
- **Enterprise/Community knowledge**: LLMs could use private data to train, so we need to design systems that can leverage this knowledge while respecting privacy and security concerns, also of our enterprise.

2 Prompting

Prompting is the process of designing inputs to LLMs in order to obtain the desired outputs. Effective prompting is essential when building AI systems because it directly affects output quality, reliability, and cost.

In this course, we focus on prompting through **API calls** rather than consumer user interfaces. The two settings differ significantly:

- **User Interfaces**: These are designed for end-users and often include features that help guide the user in crafting effective prompts, such as templates, examples, and interactive feedback. They usually don't allow you to access the model directly, so you have less control over the prompting process, but the interface itself can help users get better results.
- **API Calls**: When using APIs, you have more control and flexibility over the prompting process, as you are talking directly to the model. You have to design the prompt yourself, and you can also manage the conversation state and context more effectively. Modern APIs may also expose built-in tools such as web search, code execution, and retrieval, which can further improve results.

In short, API prompting is developer-centric, while user interfaces are user-centric. For instance, obtaining structured output can be difficult in many interfaces but straightforward through an API call.

2.1 Anatomy of a Prompt

A prompt typically consists of the following components:

- **System Prompt**: Sets the persona, rules and constraints the model follows throughout the conversation (job description). Conventionally, the system prompt is the same across all calls, but needs to be repeated since the API is stateless.
- **User Prompt**: The actual task or question you want the model to address. It can include the input data, examples and instructions on how to format the output. Conventionally, the user prompt is the part that changes across calls, depending on the specific task or input data.
- **Parameters**: These are settings that influence the model's behavior, such as temperature, top-p, max tokens, and stop sequences.

2.2 API Parameters

When making API calls to an LLM, you can specify parameters that influence model behavior:

- **Temperature**: Controls output randomness. Higher values (e.g., 0.8) usually produce more diverse and creative text, while lower values (e.g., 0.2) usually produce more focused and consistent text. In practice, the model samples from token probabilities, and temperature shapes that sampling.
- **Top-p**: Controls diversity by sampling only from the smallest set of tokens whose cumulative probability reaches `p`. It is typically used as an alternative to temperature; in many cases, tuning one is enough.
- **Max Tokens**: Caps response length, helping control both cost and verbosity.
- **Stop Sequences**: Define one or more strings that cause generation to stop when encountered.
- **Frequency Penalty**: Penalizes new tokens based on their existing frequency in the text so far, reducing repetition.
- **Presence Penalty**: Penalizes new tokens based on whether they appear in the text so far, encouraging the model to talk about new topics.

Beyond these parameters, other choices also influence output quality, especially **model selection** (for example, different model families) and **API type**.

It is useful to distinguish between **completions** and **responses**: completions are generally **stateless** (each call starts fresh unless you resend context), while responses can be **stateful**, with server-side conversation state that can simplify multi-turn interactions.

2.3 Prompting Strategies

This section covers common prompting strategies that improve model performance. Calibrating specifically is a key part of the design process, as overspecifying or underspecifying the prompt can lead to suboptimal outputs. Cutting redudancy and keeping a good structure permits to get better results, since the model follows literally the instructions

The best strategy to get a good prompt is to iteratively test and refine it, starting with a simple version and gradually adding complexity as needed.

2.3.1 Prompt Structure

Defining a clear **scope** for the model is crucial to prevent it from going off-topic or generating irrelevant information. To better handle complex tasks, it's also important to use **structure formatting** in the prompt, which can help guide the model's attention and improve the quality of the output. An example is to separate instructions, input data and output format into distinct sections, using clear headings or bullet points.
The suggested structure for a prompt is the following:

- **Role (optional)**: Define the role or persona the model should adopt (for example, "You are a helpful assistant"). This differetiates the **direct instruction** prompting strategy, where you simply provide clear, literal instructions for simple tasks, from the **role prompting** strategy, where you shape the tone, vocabulary and depth of the response.
- **Goal**: Clearly state in one sentence the overall goal or task (for example, "Summarize the following text in one sentence"). (This last part doesn't improve factual correctness)
- **Context**: Provide any necessary background information, data, or examples that the model needs to complete the task.
- **Constraints**: Specify any rules like audience, length limits and tone.
- **Output Format**: Define the expected format of the output (for example, "Return a JSON object with the following fields...").
- **Verify**: Ask the model to check its own output for errors or inconsistencies, which can help improve reliability.

Also **where you place instructions** relative to data significantly affects the quality, especially with long data. The common rule is to place long data first, instructions last and query at the very end.

2.3.2 Few-shot prompting

Few-shot prompting involves providing the model with a few examples of the desired output format or behavior within the prompt itself. This can help the model understand the task better and generate more accurate responses.
It's important to choose diverse examples covering typical cases and edge cases, and keep a consistent format across examples to help the model learn the pattern. Also ordering examples matters: place the most similar examples last, closer to the actual task, as the model tends to pay more attention to recent context.

2.3.3 Chain-of-thought prompting

LLMs generate tokens sequentially. When you force intermediate reasoning steps, each subsequent token is conditioned on better context. In other words, the model "thinks" by writing, and by prompting it to write down its reasoning process, you can often improve the quality of the final answer, especially for complex tasks that require multiple steps of reasoning.
There are several variations of this strategy:

- **Zero-shot CoT**: You simply ask the model to explain its reasoning process before giving the final answer, without providing any examples.

- **Few-shot CoT**: You provide examples of the reasoning process along with the final answer, which can help the model learn how to structure its reasoning.
- **Structured CoT**: You explicitly define the intermediate steps or subtasks that the model should follow, which can help guide its reasoning process more effectively.

3 Testing

As in the classic software design process, testing is a crucial step in building AI systems, but it also has unique challenges and considerations due to the nature of LLMs and their non-deterministic outputs.
The usual testing process involves the following steps:

- **Define Test Cases**: Identify a set of inputs and expected outputs that cover a range of scenarios, including edge cases and common use cases.
- **Analyze Outputs**: check the failing tests and iterate on the source code or prompt to fix them.

However, when testing AI systems, we need to consider additional factors:

- **Problem Complexity**: usually AI systems are designed to handle complex problems, where no single correct answer exists, and the value comes from handling ambiguity and generating creative solutions. In such cases, defining expected outputs can be challenging;
- **Overfitting to Test Cases**: If you design your prompt to pass specific test cases, you might end up with a prompt that performs well on those cases but fails to generalize to new inputs.

We can think of different approaches to tackle these challenges and improve the testing results of the system. In the following sections, we will explore some of these approaches.

3.1 Single Shot Prompts

The first idea is to use single shot prompts, where the failing test cases are directly included in the prompt as examples of what not to do, or as examples of the desired output format. This can help the model understand the specific requirements and constraints of the task, and improve its performance.

However, this approach can also lead to overfitting if the prompt becomes too tailored to the specific test cases, and it may not generalize well to new inputs, leading to terrible performance when put in production.
A general rule of thumb is to **NEVER include test cases in the prompt** and to never design the prompt to pass specific test cases. The best strategy would be to design the prompt without even knowing what the test cases are.

3.2 Self-Critique

Another approach is to use self-critique, where, in the same prompt, the model is asked to evaluate its own output and identify any errors or inconsistencies. This can help improve the reliability of the output, as the model can catch and correct its own mistakes.

A better approach is to use a separate prompt for self-critique (if possible, also with a different model), where the model is given the original output and asked to evaluate it and to provide a revised version of the output if necessary. Using a separate prompt allows the model to focus solely on the critique process, removing all the previous context and potential biases from the original prompt.

3.3 Feedback Loops

A more advanced approach is to use feedback loops. You can design a system where the model generates an output, then evaluates it against the test cases, and iteratively refines the system prompt to improve performance. There are different strategies to implement feedback loops.

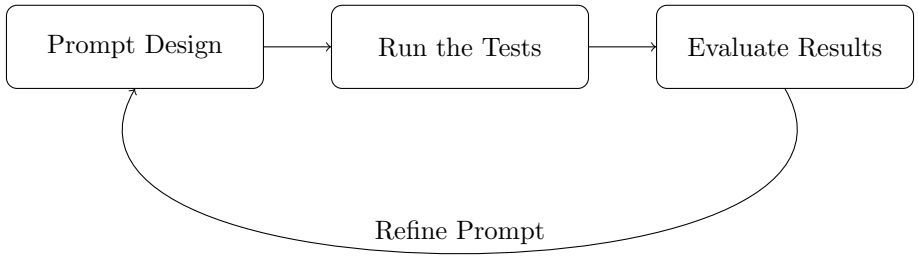

Figure 1: Feedback loop for prompt testing and refinement.

3.3.1 Human in the loop

In the basic implementation of feedback loops, a human is involved in three steps of the process:

- **Prompt Design**: The human designs the initial prompt based on their understanding of the task and the requirements;
- **Evaluate Results**: After executing the prompt on a set of test cases, the human evaluates the results and identifies any errors or inconsistencies;
- **Refine Prompt**: Based on the evaluation, the human refines the prompt to improve its performance.

We can see how this process can be time-consuming and inefficient, especially if the prompt requires multiple iterations to achieve the desired performance. Not only from a time perspective, but also from the knowledge perspective: the human needs to understand the model's behavior and how to design effective

prompts, which can be a complex task in itself.

Replacing human evaluation with an LLM judge is the key step to make the loop scalable.

3.3.2 Automatic Feedback Loops

In a fully automated feedback loop, the model itself is responsible for evaluating its own output and refining the prompt accordingly. They are based on the idea of using **LLMs as judges**, where the human is replaced by the model, which is given a set of test cases and asked to evaluate its own output against those cases. It is then asked to provide feedback on how to improve the prompt, and to generate a revised version of the prompt itself, leading to the automatic loop process.

LLM as a Judge Defining a good prompt for the LLM judge is not trivial, and it is crucial to ensure an improved performance of the system. Usually we are interested in defining a score for each parameter we want to evaluate. Based on this, we can also filter out the outputs that do not meet a certain threshold, and manually review them to understand the weaknesses of the system.

Since its cruciality, the model used for this task should be the best available, and it should be different from the one used for the original prompt, to avoid bias and to ensure a more objective evaluation.

Prompt Improvement The prompt improvement step is also crucial, as it is the one that allows the system to learn and improve over time. One of the common approaches is to ask the model to provide many variations of the original prompt (following known prompting strategies), and then to evaluate those variations against the test cases, selecting the best performing one as the new prompt for the next iteration.
Supposing that the Judge is correctly evaluating the outputs, this process can lead to a significant improvement in performance over time.

Wrong approach - Vibe Eval A common mistake is to not implement the Judge, but to have a person testing a few cases manually to check the quality of the system, and to rely on their "vibe" to decide whether the system is good or not. This approach is obviously not scalable, and it also tends to produce prompts that overfit to the specific test cases.

Avoid Overfitting The automatic approach can still lead to overfitting with small enough test sets, or after a certain number of iterations. It's important, after each iteration, to increase the size of the test set, to ensure that the system is not just learning to pass a small set of cases, but is actually improving its generalization capabilities.

Technical Suggestions In practice, when implementing feedback loops, it is important to consider the following technical suggestions:

- **Use a separate model for the Judge**: This helps to avoid bias and ensures a more objective evaluation of the outputs. The best available model should be used for this task, as the quality of the evaluation is crucial for the improvement of the system.
- **Define clear evaluation criteria**: The prompt for the Judge should include clear criteria for evaluating the outputs. Also, it should be designed as the first step of the process;
- **Batch testing**: Usually testing is done in batches, where a set of test cases is evaluated at once. There are different strategies to implement batch testing:
  - **Trust the Judge**: You can simply ask the Judge to provide a list of elements as output, and then match it with the expected outputs assuming they keep the same order. This is the simplest approach, but even stronger models can make mistakes in this type of tasks, leading to incorrect evaluations.
  - **Use IDs**: A more robust approach is to assign an ID to each test case, and ask the Judge to repeat the ID in the output. This way, we can always map the output back to the original test cases. It's important to also use short IDs, so that the models sees them as a single token.

4 Stateful Agents

The models discussed thus far are **stateless agents**. This means they do not maintain a memory of previous interactions; instead, they treat every request as an independent event and respond based solely on the current input.

To illustrate this, imagine you have access to an infinite sequence of *superhumans*. You approach each one and ask exactly one question. While each superhuman provides a brilliant response to that specific query, they possess no knowledge of your previous conversations with their predecessors. Every time you move to a new person, the context is reset to a blank slate. This behavior characterizes a stateless system.

In contrast, a **stateful agent** retains information across multiple turns. It stores previous inputs and outputs—often referred to as the *conversation history* or *context*—allowing it to understand follow-up questions and maintain continuity in a dialogue.

In this section we will explore various techniques used in agents design, from simple stateless agents to more complex stateful ones.

4.1 Stateless

As mentioned, stateless agents do not maintain any memory of past interactions. The `chat completition` api is an example of a stateless agent by design.

```
# Turn 1: tell the model your name
r1 = client.chat.completions.create(
  model="gpt-4.1-mini",
  messages=[{"role": "user", "content": "My name is Alice."}],
)
# Turn 1: "Nice to meet you, Alice!"

# Turn 2: ask for your name  but we DON'T send turn 1
r2 = client.chat.completions.create(
  model="gpt-4.1-mini",
  messages=[{"role": "user", "content": "What is my name?"}],
)
# Turn 2: "I don't have access to your name."
```

The model can't answer because it literally never saw the first message.

Although this type of agents seems pretty useless, they can be fine for single-shot tasks, like writing a poem, generating an image, or answering a question.

4.2 Default Stateful Agents

Open AI `Response API` can be used to create stateful agents.

```
resp1 = client.responses.create(model="gpt-4.1-mini", input=
    "My name is Alice.")

resp2 = client.responses.create(
    model="gpt-4.1-mini",
    input="What is my name?",
    previous_response_id=resp1.id,    # server remembers turn
        1
)
# "Your name is Alice."
```

This is convenient, but it's a black box approach. We have no control over what the model remembers and how it uses that information.

All the next techniques we will manage the state ourselves, giving us more control and flexibility.

4.3 Full History

A first approach to create a stateful agent is to send with each request the full conversation history. This is obviously the most straightforward way to maintain context, but it can quickly become inefficient as the conversation grows longer. In fact, each time we send a new message, the number of tokens we need to send increases quadratically, since we need to resend all previous messages.

This is not only inefficient in terms of latency and cost, but it can also lead to fulling the model's context window, which can cause lower quality responses or even errors.

4.4 Sliding Window

To mitigate the issues of the full history approach, we can use a **sliding window** technique. Instead of sending the entire conversation history, we only send the most recent N messages, where N is a predefined window size. This way, we can maintain some context while keeping the number of tokens manageable and constant over time.

The downside of this approach is that we might lose important information from earlier in the conversation, which can lead to less coherent responses if the model relies on that information.

4.5 Summarization

To address the limitations of the sliding window technique, we have to find a way to retain important information from the conversation without sending the entire history. This is where **summarization** comes into play. The idea is to periodically summarize the conversation history and send the summary along with the most recent messages. This way, we can keep the context manageable while still retaining important information from earlier in the conversation.

The summarization is usually done by the model itself. Along with the user message, we can provide the model with the current summary and ask it to update the summary based on the new message.

This way, the memory stays bounded, without the loss of information.

A few tips for effective summarization prompts:

- Use a structured format for the summary, organizing information into categories (e.g., "Facts", "Opinions", "Actions", etc.);
- Set a maximum length for the summary to ensure it remains concise and focused on the most relevant information;
- Resolve contradictions in the summary by asking the model to identify and address any conflicting information.

4.6 Long-Term Memory

Every technique we have seen so far is based on the idea of a *working memory*, that dies whenever the conversation ends. However, in many applications we want to have a *long-term memory* that persists across multiple conversations and sessions.

This is usually achieved by:

- Once the conversation ends, extract important information. Ask the LLM to pull out structured facts worth remembering: facts, preferences, decisions, expertise, etc;
- Store this information in an external database;
- When a new conversation starts, retrieve relevant information from the database and inject it into the prompt.

This is the actual pattern used behind the OpenAI `ChatGPT` memory feature.

4.7 When to use which pattern

Each strategy can be used in different scenarios, depending on the specific requirements of the application and the constraints of the model being used.

- **Stateless**: suitable for single shot tasks (e.g., translate, classify);
- **Full History**: suitable for short conversations;
- **Sliding Window**: suitable for longer conversations where only the most recent context is relevant;
- **Summarization**: suitable for long conversations where important information needs to be retained without exceeding the context window;
- **Long-Term Memory**: suitable for applications that require personalization or have returning users, where information needs to persist across sessions.

5 Experiments

In this section, we will discuss the basic recipe for "evaluating" in **software 2.0** or more in general in machine learning.

We'll mostly focus on binary classification problems, where the goal is to classify inputs into two categories (for example, "spam" vs "not spam", or "positive" vs "negative"). Usually the metrics used is the proportion of misclassified inputs (also known as **error**), meanwhile the proportion of correctly classified inputs is called **accuracy**.

5.1 Comparing Models

Usually, before putting a model into production, we want to compare it with other models to understand which one performs better on a given task. We could compare different model types but also different parameter settings for the same model type.

In simple cases, we can in a 2D dataset plot the predicted output and visually inspect the decision boundary to understand, but this approach is not scalable to high-dimensional datasets. Here, the most straightforward approach, is to training both models on the same training set, and pick the one with fewer mistakes. This generates a few questions:

- On which data we compute the error?
- How we eliminate random effects?
- Are the metrics we are using good?

5.2 Test Sets

As we have seen before, **never judge your performance on the training set**, since the model has been trained to minimize the error on that specific set, and it may not generalize well to new inputs.

So the first thing we do in machine learning is to split the dataset into a training set and a test set. This should allow us to suppose that the test set is representative of the real-world data distribution.
So in this setting, becames crucial to perform a good split of the dataset. We don't split the dataset based on proportion, but in the **absolute size** of the test set, which should be large enough to be representative. Ideally, we want 10000 examples in the test set, but this is not always possible, especially for small datasets.

5.2.1 Overfitting in the test set

However, this approach can lead to overfitting on some cases. Suppose we want to choose the best hyperparameters for our model, and we are provided with a test set. We try different hyperparameter settings, and we pick the one that performs best on the test set, supposing that it will generalize well to unseen data. But, at a certain point, we are provided with a fresh test set, and we find out that the performance is much worse than expected with the chosen hyperparameters. This is because we have **overfitted** on the test set.

Our method for choosing the best hyperparameters are another learning algorithm, and by testing different hyperparameters on a small test set, leads us to fit on **random fluctuations** in the test set, which may not be representative of the real-world data distribution.

To simple answer to this problem is **not to test multiple times on the test set.**

5.2.2 Validation Sets

Reusing the test set not only makes you choose the wrong model, but also **inflates the performance measures**, making them lower than they actually are.

To avoid this you need to test which model to use, which hyperparameters to give it, and how to extract features only on the training data. In order not to do this all on the training set, we can split the training set into a **training set** and a **validation set**. Ideally, the validation set is the same size of your test set, but you can make it a little smaller to get more data for training.

Note that this approach by itself does not solve the problem of preventing multiple testing on the test set. It just provides for a final failsafe to detect it, but you should first convince yourself that the results you see are not down to multiple tests. You need always to be careful and whenever it's needed to rerun your experiments on training and validation sets.
However, if your dataset is large, and you haven't done anything strange in tuning phase, the results between test set and validation set should be similar.

5.2.3 Cross Validation

Doing this split could lead to very small sets, especially if the original dataset is small. In this case, we can use **cross validation**, which is a technique that allows us to use the entire dataset for both training and testing, by splitting it into multiple folds. For example, in a 5-fold cross validation, we split the dataset into 5 equal parts, and we train the model on 4 parts and test it on the remaining part (validation). We repeat this process 5 times, each time using a different part as the test set,and we average the results to get a more reliable estimate of the model's performance.

This can be costly, but ensures that every instance in the dataset is used for training. After chosen your model and hyperparameters through the cross-validation process, you can test it once on the test set to get an estimate of its performance on unseen data.

5.2.4 Temporal Data

If your data has a temporal ordering, you need to take this into account when splitting the dataset. In this case, you can't sample your test set randomly, since training on samples that are in the future compared to the test set is unrealistic. You need to maintain the ordering, and split the dataset based on time, using the most recent data as the test set, and the older data as the training set.

However, if your data has a timestamp, but there's no meaningful information in the ordering (like in email classification) seeing from the future doesn't usually give you much of an unfair advantage in the task.

If you want to apply cross validation to temporal data, that takes a small portion of initial data as the training set and then iteratively adds more data to the training set, while validating on the next portion of data, until you reach the end of the dataset.

Final remark: Philosophically talking, when in doubt make sure that the evaluation setting accurately simulates production, and that the validation setting accurately simulates the evaluation setting,

5.3 Experiment Design

Now that we now how to experiment, what experiments should we run? Which hyperparameters should we test? Basically as long as you are not looking at the test set, you can do whatever to like.

5.3.1 Hyperparameter Tuning

When tuning hyperparameters, we have the following options:

- **Trial and Error**: This is the most basic approach, where you simply try different combinations of hyperparameters and see which one performs best on the validation set. This is driven by intuition and experience, and it can be time-consuming, especially if you have a large number of hyperparameters to tune.
- **Grid Search**: This is an automated approach, where you define a grid of relevant hyperparameter values, and the algorithm trains and evaluates a model for each combination of hyperparameters in the grid. This can be

computationally expensive, especially if the grid is large, but it ensures that you are exploring a wide range of hyperparameter combinations.

- **Random Search**: This is another automated approach, where you randomly sample combinations of hyperparameters from a defined distribution, and train and evaluate a model for each combination. This can be more efficient than grid search, especially if you have a large number of hyperparameters, since it allows you to explore a wider range of combinations.

6 Statistical Testing

ML and statistics are closely related, but significance testing is rarely used in ML experiments due to large sample sizes and emphasis on replication over statistical significance. Replication is key, but often challenging in practice, so we can still use statistical testing as a tool to assess the reliability of our results.

6.1 True metrics vs Estimated metrics

When we evaluate a model, we usually imagine that data a sampled from $p(x)$, and we want a classifier that minimizes the error on the true distribution, not the one we have in our dataset. However, we can only compute the statistic on the dataset, which is called the **sample metric**. All metrics have a true value defined on the true distribution, but we can only compute an estimate of it based on the data we have.

This brings to us the need to prove how much near the sample metric is to the true metric, and that is not just a noise.

6.1.1 Confidence Intervals

One way to do this is to compute confidence intervals, which are ranges of values that are likely to contain the true metric with a certain level of confidence. This helps us understand the uncertainty around our estimate and make more informed decisions about model performance.

This can be done using various test samples, making the sample metric a random variable. This makes the sample metric vary across different samples, and we can use this variability to compute confidence intervals.

The size of the confidence interval depends on two factors:

- **True metric**
- **Sample size**

If you have not the luxury of a large sample size, you can do some statistical testing to understand how much the sample metric is close to the true metric.

Remark: Since we are working with estimated value, also the confidence interval is an estimate, and it is not guaranteed to contain the true metric with the specified confidence level. So it relative to the test sample we have, and its important to understand that the confidence interval is not a fixed range, but it can vary across different samples.

Standard Deviation and Standard Error: The standard deviation of the sample metric is a measure of how much the sample metric varies across different samples. The standard error is the standard deviation of the sample mean, which is the sample metric in this case, and it is computed as the standard deviation divided by the square root of the sample size.

The standard deviation is the measure of **spread** of the sample metric, meanwhile the **confidence interval** and standard error are measures of **uncertainty** around the true metric. This makes the two concepts different.

6.2 Randomness

In ML we know that, the results of an experiment can completely change by rerunning due to the **random initialization** of the model. Also **data sampling** can introduce randomness, due to the fact that we are working with a sample of data, and not the entire population.

This is why we need to evaluate how much the results are robust to randomness. For random initialization, it's enough to run the experiment multiple times, but how we test the robustness to data sampling?

6.2.1 Resampling Methods

The basic approach is to analyze the **standard deviation** of the sample metric across different samples, and to compute confidence intervals for the true metric. Through cross-validation, we can compute the sample metric on different subsets of the data, and analyze how much it varies across these subsets. This allows us to assess the robustness of our results to data sampling, and to understand how much the sample metric is likely to vary if we were to collect a different sample of data.

However, as we increase the number of repeats, we are incrementing the overlap between the samples, especially if we are working with a small dataset.

6.2.2 Bootstraping Methods

An alternative approach is to use bootstraping methods, in which we **resample with replacement** from the original dataset to create multiple bootstrap samples. This allows us to sample a dataset that is exactly the same size as the original dataset, permitting us to resample as many times as we want without worrying about overlap between samples.
This method approximates the data distribution in a very precisely way.

6.3 Evaluating Regression

When computing a continuous metric, such as the mean squared error m, we assume individual observations are distributed around a true metric. The sam-

ple mean (computed from the results of the testing) serves as a point estimate of this expectation, but it is itself a random variable. Across different samples, the sample mean follows a Student's t-distribution which converges to a normal distribution for sample sizes $n > 30$, centered at the true mean. This allows us to compute confidence intervals for the true metric based on the sample mean and its variability.

6.3.1 Metrics

The most common metrics for regression is the mean squared error (MSE), which measures the average squared difference between the predicted values and the true values. However, also RMSE (root mean squared error) is used, which is the square root of MSE, and it is more interpretable since it is in the same units as the original data, instead of being in squared units.

6.3.2 Bias and Variance

Bias and variance are two sources of error in regression models. Those definitions are strictly related to the MSE, but they can be extended to other metrics as well.

- **Bias** It's the difference between your optimal MSE and the true MSE, and it is a measure of how much your model is underfitting the data. This part remains the same also if you resample the data, since it is a property of the model itself, and not of the data.
- **Variance** It's the difference between the true MSE and the measure (across samples), and it is a measure of how much your model is overfitting the data. This part can vary across different samples, since it is a property of the data, and not of the model itself.

Often the bias and variance are in a trade-off, since increasing the complexity of the model can reduce the bias but increase the variance, and vice versa.

6.4 Evaluating Classification

When evaluating classification models, we often use metrics such as accuracy, precision, recall, and F1-score. These metrics are based on the confusion matrix, which counts the number of true positives, true negatives, false positives, and false negatives.

6.4.1 Class Imbalance

It can happen that the classes in the dataset are imbalanced, meaning that one class is much more frequent than the other. In this case, getting a accuracy of 90% can be **misleading**, since the model can simply predict the majority class and still achieve a high accuracy.
In production, this can lead to a model that performs poorly on the minority

class, which can be critical in certain applications, such as fraud detection or medical diagnosis.

Another problematic case, could be the one where all the minority class is contained in the test set, and the model is not able to learn anything about it, since it is not present in the training set. This can lead to a model that performs very poorly on the test set, and it is not able to generalize to new data.

6.4.2 Cost Imbalance

In some cases, the cost of misclassifying one class can be much higher than the cost of misclassifying the other class. For example, in medical diagnosis, misclassifying a sick patient as healthy can have much more severe consequences than misclassifying a healthy patient as sick.
In these cases, it's important to use metrics that take into account the class imbalance and cost imbalance. We can tune our error by multiplying the error of each class by a cost factor, which can be determined based on the specific application and the relative importance of the classes.

It's important to note that, in both cases, the cost is never zero, even in the example of medical diagnosis, since misclassifying a healthy patient as sick can lead to unnecessary treatments and anxiety for the patient. Especially in social applications, where could bring to a lot of ethical issues.

6.4.3 Confusion Matrix

The confusion matrix is a table that summarizes the performance of a classification model by counting the number of true positives, true negatives, false positives, and false negatives. This allows us to compute various metrics.

However it's difficult to compare two models based on the confusion matrix, but it can give good insights on the single model works. The problems previously described affects the diagonal of the confusion matrix (false positives and false negatives - misclassifications)

You can plot the confusion matrix both for the training set, validation set, and test set, giving different insights on the three sets.

6.4.4 Precision and Recall

Precision is the ratio of true positives to the total number of predicted positives, while **recall** is the ratio of true positives to the total number of actual positives. In order to handle class imbalance, we need to trade-off between precision and recall, since increasing precision can lead to a decrease in recall, and vice versa.
To help us in this trade-off, we can plot the precision and recall for different classifier.

6.4.5 TPR and FPR - ROC space

TPR (True Positive Rate) is the ratio of true positives to the total number of actual positives, while **FPR** (False Positive Rate) is the ratio of false positives to the total number of actual negatives. We want to maximize TPR and minimize FPR, but as before there is a trade-off between the two. The optimal space is TPR/FPR = 1, which means that we are maximizing TPR and minimizing FPR at the same time, this is called the **ROC space** (Receiver Operating Characteristic).

6.4.6 Plotting thesholds

Until now we have been talking about the comparison of different models, but we can also compare the same model with different thresholds (eager to predict positive). If we have a shy model (high threshold), we would not predict anything, however if we have a bold model (low threshold), we would predict everything as positive even risking misclassifications. The curve that we have, it can be a good way to compare the performance of the model across different thresholds, and to choose the optimal threshold based on the specific application and the trade-off between precision and recall.

However, this is possible only if the model outputs is **ranking classifier** (i.e., it provides a score for each instance), which is not always the case, since some models output a binary prediction without a score. In this case, the **ranking error** is the error measured in confidence, if we predict a positive class with a high confidence, but it is actually a negative class, this would be an high ranking error.

6.4.7 Coverage Matrix

Given a model that outputs a ranking/score, we can plot the coverage matrix, with negative class on the x-axis and positive class on the y-axis. Basically the more confident the model is predicting a positive class, the closer the point is to the top right corner, while the more confident the model is predicting a negative class, the closer the point is to the bottom left corner. Then coloring binary the points based on the predicted class (after thesholding), we can have a good visual representation of the performance of the model.

6.4.8 AUC

The AUC (Area Under the Curve) is a metric that summarizes the performance of a classification model across all possible thresholds. It is computed as the area under the ROC curve, and can be used to compare different models. If we normalize the coverage matrix, we get the ROC curve, and the AUC is the area under this curve. The AUC (in ROC space) is an estimate of the probability that a ranking classifier puts a randomly drawn pair of positive and negative examples in the correct order.

Lets now look how to works for another type of classifier. The decision tree is an example of a **partitioning classifier**, and how we get a ranking from a classifier that depends entirely on the model class, in other words, it depends on the way the model is built.
If we choose to use the proportion of positive point for each segment (leaf) as the score, and we then put all the points in one region on the same level in the ranking. This means that some pairs of points will have the same score, and this can lead to ties in the ranking. Then, for large datasets, these regions will not contribute much to the total curve.

6.4.9 Precision-Recall Curve

The Precision-Recall curve is another way to evaluate the performance of a classification model, similar to the ROC curve, but in the axis we have precision and recall instead of TPR and FPR. The area under the Precision-Recall curve (AUC-PR) can be used as a metric to compare different models, especially in cases of class imbalance, where the ROC curve can be misleading.
However in some cases, the Precision-Recall curve can be more difficult to interpret than the ROC curve, since it is not normalized and it can be affected by the class imbalance in a different way than the ROC curve. Both curves can be useful in different scenarios, it's suggested to plot both.

6.4.10 Use of Metrics

There are many others metrics that can be used to evaluate classification models, and they can be applied to all the sets (training, validation, and test set). For instance, if we compute the accuracy on the test set, we are talking about the **test accuracy**, used to show if our conclusions are valid. Meanwhile, the **validation accuracy** is used to tune the hyperparameters of the model. And the **training accuracy**, which is used in comparison with the validation accuracy (difference **generalization gap**) to understand if the model is overfitting or underfitting the data. It's important to note that it's not used for evaluating on the training set.

6.5 No Free Lunch Theorem

A question that often aries is: which classifier, model, search methods is the best? The aim is to make a guess before seeing data, but the No Free Lunch Theorem states that is not possible.

If you average over every possible problem, no optimization or learning algorithm is better than any other. An algorithm only looks good when the problem happens to match its assumptions. This means that there is no universally best model or algorithm. (**problem of induction**)

The **inductive bias** of a learning algorithm or model are those assumptions about the domain that are explicitly or implicitly hardcoded into the model. In other words, the inductive bias is the set of assumptions that a model makes about the data in order to generalize from the training data to unseen data. As a computer scientist, we need to figure out which of the many possible inductive biases is the best for our problem. For instance a linear model has a strong inductive bias towards linear relationships between features and the target variable.

Finally, when two models explain the data equally well, we usually prefer the simpler one. The intuition is that simple patterns are more likely than very complex ones, so learning methods often include a built-in **bias toward simplicity**.